



## Interrogación 3

11 de diciembre de 2025

**Tiempo:** 2 horas

**Condiciones de entrega:** Se deben entregar 3 de las 4 preguntas.

**Forma de Entrega:** La entrega es a través de una imagen digitalizada en Canvas. Al final de la prueba tienen 10 minutos para subir la imagen de la prueba y el torpedo, en formato vertical, cada pregunta por separado.

**Dispositivo:** Para digitalizar la prueba se permite el uso de UN dispositivo (teléfono, tablet) el cual debe estar guardado y en silencio durante la prueba.

**Torpedo:** Se permite una hoja tamaño oficio escrita a mano.

**Evaluación:** Cada pregunta tiene 6 puntos (+1 punto base). La nota de la interrogación es el promedio de las 3 preguntas entregadas

**Salida de la sala:** Si necesita salir de la sala al baño, debe dejar el celular adelante en la mesa con los ayudantes.

*Lea todas las preguntas antes de iniciar las respuestas.*

### 1. Rutas más cortas

Para encontrar la mejor ruta entre origen y destino Waze usa un algoritmo de camino más corto en grafos, donde cada arista tiene un costo base  $\text{costo}(u,v)$  no negativo ( $\text{tiempo} = \text{distancia} / \text{velocidad actual}$ ), sin embargo, los costos de las próximas aristas a recorrer tienen costos dinámicos que cambian según el tráfico, incidentes y restricciones viales actuales.

1. (2 pts.) Modele como un grafo el problema de ir de un punto a otro en una ciudad (sin carreteras, rotondas, etc.) consistente en una cuadrícula con cuadras y calles, indicando de forma justificada si el grafo es direccional, ¿que representan los Vértices, Aristas y Costos?.
2. (4 pts.) Tomando como base el algoritmo de Dijkstra, genere un nuevo algoritmo en pseudocódigo  $p = \text{waze}(i, f)$  que, a medida que ud. avanza por el grafo va encontrando el camino más corto al punto de destino y lo va almacenando en  $p$ . La condición inicial  $P = \text{waze}(0, n)$  donde el punto de origen es 0 y el final  $f$ . Al avanzar el algoritmo va adaptándose a las nuevas condiciones desde el punto actual al final.

#### 1.1. Pauta

##### 1. Modelo

- Vértices: Intersección de calles, esquina, entrada o punto relevante (incluyendo incidentes, origen 0 y destino  $f$ ).
- Aristas: tramo de calle entre dos intersecciones consecutivas. Es un grafo dirigido según la dirección del tránsito, doble sentido son dos aristas puestas direccionales contrarias.

- Costos de arista: el costo asociado a una arista es el tiempo de viaje estimado:  $\text{costo}(u,v) = \text{distancia}(u,v) / \text{velocidad actual}(u,v)$ , que se actualiza dinámicamente según tránsitos, incidentes y restricciones (cortes, tramos cerrados, etc.)

2. El algoritmo debe tener 3 componente principales, si estan presentes está bien

- Debe haber un ciclo que itere por cada nodo que se avanza
- Ejecutar Dijkstra modificado para que entregue dist y padre en cada ciclo
- reevaluar las siguientes aristas en cada ciclo. `actualizarCostosDinámicos()`

con esto tenemos:

```

    reconstruye el camino una vez ejecutado Dijkstra
función reconstruirCamino(origen, destino, padre):{
    camino ← lista_vacia()
    x ← destino
    mientras x nulo:
        insertarInicio(x, camino)
        x ← padre[x]
    retornar camino    // [origen, ..., destino]
}
función waze(i, f):
    inicializacion de variables
    p ← null
    actual ← i
while actual f do{
    ejecute dijkstra normal, con costos actuales modificado para entregar las distancias PASO IMPORTANTE
    (dist, padre) ← dijkstra(actual) PASO IMPORTANTE
    dist contiene las distancias desde actual a todos los nodos
    padre contiene el padre de cada nodo en el camino desde actual
    camino ← reconstruirCamino(actual, f, padre)
    siguiente ← camino[1] vecino
    agregarFinal(actual, p)
    \avanzar" una cuadra
    actual ← siguiente
    tráfico, incidentes, etc.
    actualizarCostosDinámicos() PASO IMPORTANTE
}
agregarFinal(f, p)
retornar p

```

En el algoritmo de Prim usamos un heap binario  $Q$  como la cola de prioridades que en cada iteración nos dice cuál de los nodos que aún no están en la solución —es decir, en el MST  $T$  que se está formando— es el nodo  $u$  más cercano a la solución (es decir, el nodo más cercano a algún nodo  $x$  que ya está en  $T$ ). Luego de que la arista  $(x, u)$  pasa a formar parte de  $T$ , el heap  $Q$  es actualizado de modo que al iniciarse la siguiente iteración nos pueda decir cuál es el nuevo nodo más cercano. [En el algoritmo visto en clases,  $[x = \pi[u] = u.parent]$  .]

Una de las gracias de usar un heap binario como cola de prioridades es que la complejidad del algoritmo es  $O((V+E) \log V)$  para un grafo  $G = (V, E)$ .

Supongamos ahora que implementamos la cola de prioridades no como un heap binario, sino simplemente como un *array*  $d[1..|V|]$ , en que  $d[i]$  es la distancia actual del nodo  $i$  a  $T$ . En cada iteración, buscamos secuencialmente en  $d$  el nodo  $u$  tal que  $d[u]$  es el menor valor del *array*, incluimos la arista  $(x, u)$  en  $T$ , y actualizamos  $d[v]$  para cada  $v$  en  $\alpha[u]$ .

¿Cuál es ahora la complejidad del algoritmo de Prim? Justifica tu respuesta tomando en cuenta explícitamente lo que cuesta encontrar el nodo  $u$  y lo que cuesta actualizar los  $d[v]$ . Puedes suponer que, cada vez que encuentras un nodo  $u$ , colocas en  $d[u]$  una marca especial para no volver a tomarlo en cuenta en las próximas iteraciones.

### Solución

El algoritmo hace  $|V|$  iteraciones: cada vértice sale una vez de  $Q$ ; y originalmente todos están en  $Q$ . [1 pt.]

En cada iteración, el algoritmo extrae el mínimo de  $Q$ : en la nueva implementación, esto significa recorrer todo el *array*  $d$  buscando el mínimo valor  $d[u] \Rightarrow$  cada iteración es  $O(V)$   
 $\Rightarrow$  en total es  $O(V^2)$ ; [2 pts]

... y dentro de  $c$ /iteración tiene que actualizar  $d[v]$  para cada  $v$  en  $\alpha[u]$  (para el  $u$  encontrado arriba); esto es, tiene que mirar el vértice  $v$  en el otro extremo de la arista  $(u, v)$  y actualizarlo (si es necesario). C/u de estas actualizaciones toma sólo  $O(1)$  y hay  $|E|$  de ellas en total ( $c$ /arista es procesada de esta manera exactamente una vez a lo largo de todo el algoritmo). [2 pts.]

Por lo tanto el algoritmo es  $O(V^2 + E)$ . [1 pt.]

## X) Bellman-Ford

El algoritmo de Bellman–Ford sirve para encontrar los caminos más cortos desde un nodo origen a todos los demás nodos en un grafo dirigido o no dirigido, incluso cuando existen aristas con peso negativo. El pseudo código de Bellman-Ford visto en clases es:

```
ValidBellmanFord(s):
1  for  $u \in V$  :
2       $d[u] \leftarrow \infty$ ;  $\pi[u] \leftarrow \emptyset$ 
3   $d[s] \leftarrow 0$ 
4  for  $k = 1 \dots |V| - 1$  :
5      for  $(u, v) \in E$  :
6          if  $d[v] > d[u] + \text{cost}(u, v)$  :
7               $d[v] \leftarrow d[u] + \text{cost}(u, v)$ 
8               $\pi[v] \leftarrow u$ 
9  for  $(u, v) \in E$  :
10     if  $d[v] > d[u] + \text{cost}(u, v)$  :
11         return false
12     return true
```

- a) [2 puntos] ¿Qué estrategia algorítmica utiliza Bellman-Ford? ¿Cómo reconoce dicha estrategia algorítmica y qué condiciones cumple el problema de caminos más cortos que permite utilizarla?

R:

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0,5 pts. | Utiliza Programación Dinámica                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 0,5 pts. | Se reconoce porque: <ol style="list-style-type: none"><li>1. Determina la solución al problema original a partir de subproblemas que se solapan</li><li>2. Recuerda las soluciones a subproblemas</li><li>3. Hay una recurrencia sencilla</li></ol>                                                                                                                                                                                                                                |
| 1,0 pts. | <ol style="list-style-type: none"><li>1. Una ruta óptima está compuesta por una sub ruta óptima anterior más la arista que se agrega (sub problemas que se solapan)</li><li>2. El costo para llegar a un nodo solo se actualiza cuando encontramos una mejor forma de llegar a él (recuerda las soluciones a los subproblemas)</li><li>3. Una arista es parte de la ruta óptima o no, la elección depende de elegir el mejor entre ambas opciones (recurrencia sencilla)</li></ol> |

- b) [2 puntos] Explique claramente por qué razón basta realizar  $|V| - 1$  iteraciones para encontrar el camino más corto desde  $s$  a los demás nodos utilizando Bellman-Ford, y por qué razón realizar la iteración siguiente,  $k = |V|$ , permite detectar ciclos de costo negativo en el grafo.

R:

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1,0 pts. | <ul style="list-style-type: none"> <li>• Si se realizan menos iteraciones que <math> V  - 1</math> un camino simple (sin ciclos / nodos repetidos) puede no llegar a todos los nodos de destino (se necesitan <math> V  - 1</math> aristas para conectar todos los nodos del grafo.</li> <li>• Luego, <math> V  - 1</math> iteraciones garantiza revisar todos los caminos simples posibles entre el origen y todos los demás nodos del grafo, de largo entre 1 y <math> V  - 1</math> aristas, identificando aquellos que entregan el menor costo.</li> </ul> |
| 1,0 pts. | <ul style="list-style-type: none"> <li>• Si se realizan más de <math> V  - 1</math> iteraciones, necesariamente se repite un nodo en el camino, el camino ya no es simple, lo que implica un ciclo.</li> <li>• Si no hay ciclos de costo negativo, el ciclo necesariamente aumenta el costo, luego la arista adicional no sería parte de la ruta óptima.</li> <li>• Por otro lado, si la arista adicional mejora algún costo, esto implica que existe un ciclo de costo negativo en el grafo.</li> </ul>                                                       |

- c) [2 puntos] Escriba el pseudo código de la función *pathBF(s)* que retorna (imprime) los caminos más cortos desde el nodo *s* a todos los demás nodos del grafo, sólo cuando el grafo no tiene ciclos de costo negativo. Puede utilizar la función *ValidBellmanFord(s)* en su solución.

|          |                                                                                                                                                                                                                                                                                                                                                                        |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2,0 pts. | <pre> pathBF(s)   if ValidBellmanFord(s)     S &lt;- stack vacío     for u in V - {s}       push(S, u)       v &lt;- pi(u)       push(S, v)       while v &lt;&gt; s         v &lt;- pi(u)         push(S, v)       while S no vacío         print(pop(S)) // imprime el stack desde s hasta u     else       print("El grafo tiene ciclos de costo negativos") </pre> |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 4. Floyd-Warshall

Considera el siguiente grafo direccional con costos, con vértices a, b, c, d y e, representado mediante sus listas de adyacencias:

a : [b, 3] – [c, 8] – [e, -4]

b : [d, 1] – [e, 7]

c : [b, 4]

d : [a, 2] – [c, -5]

e : [d, 6]

Algoritmo de Floyd-Warshall

D = matriz de adyacencias

for k = 1 ... n |n es el número de vértices

  for i = 1 ... n |para cada vértice i

    for j = 1 ... n |para cada vértice j

      dij = min( dij, dik+dkj )

return D

- (2 pts.) Ejecuta el algoritmo de Floyd-Warshall sobre el grafo anterior; muestra el contenido de la matriz D de costos menores después de cada iteración del índice k, y encuentra tanto las longitudes de las rutas más cortas como las rutas propiamente tales.

$$\begin{aligned}
 D^{(0)} &= \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(0)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
 D^{(1)} &= \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(1)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
 D^{(2)} &= \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(2)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
 D^{(3)} &= \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(3)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
 D^{(4)} &= \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} & \Pi^{(4)} &= \begin{pmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix} \\
 D^{(5)} &= \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} & \Pi^{(5)} &= \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}
 \end{aligned}$$

- (2 pts.) ¿Cómo podemos usar el resultado del algoritmo de Floyd-Warshall para detectar la presencia de un ciclo con costo acumulado negativo? Justifica.

Respuesta: Una posibilidad es ejecutar una iteración adicional, con  $k = n+1$ , y ver si alguno de los valores de la matriz D cambia. Si hay ciclo con costo acumulado negativo, entonces el costo de alguna ruta más corta deberá disminuir más allá del "mínimo" encontrado después de las  $n$  iteraciones. Lo otro es mirar los valores de la diagonal de D. Hay un ciclo con costo acumulado negativo si y sólo si alguno de esos valores es negativo.

3. (2 pts.) Este algoritmo es un algoritmo de programación dinámica; muestra que es aplicable al problema: Demuestra que el problema tiene subestructura óptima. Respuesta: Cualquier camino óptimo desde  $i$  a  $j$  usando vértices hasta  $k$  es óptimo si el subcamino desde  $i$  a un intermedio  $m \leq k$  y de  $m$  a  $j$  también lo es. Específicamente:

$$d_k(i, j) = \min(d_{k-1}(i, j), d_{k-1}(i, k) + d_{k-1}(k, j))$$

Esto se debe a que, si un camino óptimo pasa por  $k$ , se descompone en dos subproblemas óptimos más pequeños sin  $k$ .

Por inducción la ecuación de recurrencia garantiza optimalidad: se asume  $d_{k-1}$  óptimos, luego  $d_k$  considera todas las opciones viables (directa o vía  $k$ ), preservando la minimalidad. Los casos base son

$$d_0(i, j) = w(i, j)$$

$$d_k(i, i) = 0$$